Report Exercise 2
Advanced Computer Architectures

()

Michiel Vandromme & Joffrey Van Cauwenberghe

February 25, 2024



Class: Master Software Engineering

1 Introduction

This second lab of GPU programming with CUDA delves into strategies for mitigating race conditions when parallellism is used. Race conditions occur when multiple threads simultaneously access shared data, and the order of access impacts the outcome. Given the GPU's reliance on parallelism and the availability of global and shared memory, meticulous attention is crucial to avoid unpredictable behavior. We'll explore three safe approaches in the context of finding the maximum value within an array: a CPU-only implementation, one utilizing atomic operations and finally a reduction-based solution.

2 Implementation

Calculating the maximum value of an array using only the CPU in C++ is quite straightforward. First, we create an array with a chosen size and initialize the maximum to negative infinity. Then, we iterate over the array, and each time the element is greater than the current maximum, we overwrite it. Using the chrono library, we can track the processing time of this approach.

In the case of the implementation with atomic operations, we refer to our first laboratory session. In that session, we learned that we need to allocate memory on the GPU for the inputArray and for the output. Subsequently, the hostArray must be copied to the allocated device memory. We define the number of threads per block (256) and calculate the number of blocks based on that. Using a cudaEvent_t, we can also track the processing time of the method that searches for the maximum. In that method, each thread takes an element from the array and attempts an atomicMax operation with the current maximum. Since only one thread can perform this atomic operation at a time, the threads must wait for each other.

Finally, we utilized reduction to find the maximum and make optimal use of the parallel capabilities of the GPU. In this approach, we employ a for-loop that combines partial results at each iteration. After each iteration in the for-loop, threads must wait for each other, as there is a possibility that a thread may not have read the latest value. This effect is achieved using the syncthreads method, which ensures that threads within one block wait at that point for each other.
In each iteration, the first s elements inside a block compare their value with the value located s positions further and replace their value with the larger of the two. After each iteration, s is divided by two. Initially, s is initialized to half the width of a block. This method ensures that at the end of each iteration, the largest values are located at the first s positions. The for-loop stops when s becomes less than 1 because, at that point, the largest element is entirely at the front.
Finally, we perform an atomicMax operation on the output value to avoid race conditions between blocks

3 Evaluation & Conclusion

If we compare our different methods in terms of execution time, we obtain the following results for an array of 800 elements:

CPU only : 0.035 milliseconden
AtomicMax: 1.6-1.8 milliseconden
Reduction: 1.8-2 milliseconden

These results seem somewhat contradictory to what we expected. The reason why the CPU is so much faster is probably due to the fact that the other 2 methods require the GPU to be initialised and here the CPU can just carry on calculating. Finding the maximum of an array is also a pretty simple job, even if the array is large. The reason we expect reduction to be slightly slower than atomicMax is because the atomicMax operation is enormously fast. Threads fight for the semaphore/mutex of the max variable as quickly as possible. With reduction, threads have to be synchronised multiple times in a for-loop which can cause the very minute overhead. Even with larger arrays (3000 bv), the difference in time between methods remains in the same order.